

Functional Dependencies & BCNF Normalization Proof

This section lists the **minimal (canonical) set of functional dependencies** for every relation and formally proves BCNF compliance.

BCNF Rule: A relation R is in Boyce-Codd Normal Form (BCNF) if and only if, for every non-trivial FD $X \rightarrow Y$, the determinant X is a superkey of R.

Since $BCNF \supseteq 3NF \supseteq 2NF \supseteq 1NF$, proving BCNF also proves all lower normal forms.

Relation	Candidate Key	Functional Dependencies	BCNF ?
FASHION_SHOW	(ShowID)	ShowID $\rightarrow$ Theme, Date, Season, Expected_Audience, Initial_Budget, VenueID, OrganizerID	Yes
VENUE	(VenueID)	VenueID $\rightarrow$ Name, City, Country, Capacity, RentalCost	Yes
ORGANIZER	(OrganizerID)	OrganizerID $\rightarrow$ FullName, Email, Phone, Username, Password	Yes
SEGMENT	(SegmentName, ShowID)	(SegmentName, ShowID) $\rightarrow$ StartTime, EndTime	Yes
DESIGNER	(DesignerID)	DesignerID $\rightarrow$ Name, BrandName, YearsOfExperience, Specialization, Country, Contact, Website	Yes
MODEL	(ModelID)	ModelID $\rightarrow$ FullName, Contact, AgencyName, Height, Language, PortfolioLink	Yes
GARMENT	(GarmentID)	GarmentID $\rightarrow$ Name, Type, Fabric, Color, ProductionCost, DesignerID	Yes
SPONSOR	(SponsorID)	SponsorID $\rightarrow$ CompanyName, Industry, ContactName, ContactEmail	Yes
MEDIA_REP	(MediaID)	MediaID $\rightarrow$ OrganizationName, MediaType, ContactPerson	Yes
GUESTS	(GuestID)	GuestID $\rightarrow$ Name, Email, Phone, VIP_Status, Profession, Gender	Yes
CREW	(CrewID)	CrewID $\rightarrow$ Name, Experience_Years, Email, Role, OrganizerID	Yes
PARTICIPATES	(DesignerID, ShowID)	(DesignerID, ShowID) $\rightarrow$ Confirmation_Status	Yes
WALKS_IN	(ModelID, GarmentID, SegmentName, ShowID)	(ModelID, GarmentID, SegmentName, ShowID) $\rightarrow$ WalkNumber	Yes
SPONSORSHIP	(SponsorID, ShowID)	(SponsorID, ShowID) $\rightarrow$ ContributionAmount, BenefitType	Yes
COVERED_BY	(MediaID, ShowID)	(MediaID, ShowID) $\rightarrow$ EstimatedReach	Yes
LANGUAGES	(ModelID, Language)	No non-trivial FDs : entire schema is the key (trivially in BCNF)	Yes
INVITED_TO	(GuestID, ShowID)	No non-trivial FDs : trivially in BCNF	Yes
WEARS	(ModelID, GarmentID)	No non-trivial FDs : trivially in BCNF	Yes
CONSISTS_OF	(CrewID, ShowID)	No non-trivial FDs : trivially in BCNF	Yes

Single-Attribute Key Relations (e.g., FASHION SHOW)

In relations such as FASHION_SHOW, VENUE, ORGANIZER, DESIGNER, MODEL, GARMENT, SPONSOR, MEDIA_REP, GUESTS, and CREW, each table has a single primary key.

Example:

- ShowID → Theme, Date, Season, Expected_Audience, Initial_Budget, VenueID, OrganizerID

Here:

- The determinant (ShowID) is the primary key
- A primary key is always a superkey

Therefore:

Every functional dependency satisfies the BCNF condition.

These relations are in BCNF.

Composite-Key Relations (e.g., SPONSORSHIP)

Some relations use a composite primary key.

Example:

- SPONSORSHIP(SponsorName, ShowID, ContributionAmount, BenefitType)

Functional Dependencies:

- (SponsorName, ShowID) → ContributionAmount
- (SponsorName, ShowID) → BenefitType

Here:

- The determinant is the full composite key
- The composite key is a candidate key → superkey

Therefore:

BCNF condition is satisfied.

Trivial BCNF Relations (LANGUAGES, INVITED TO, WEARS, CONSISTS OF)

Some relations contain no non-trivial functional dependencies.

Examples:

- LANGUAGES (ModelID, Language)
- INVITED_TO (GuestID, ShowID)
- WEARS (ModelID, GarmentID)
- CONSISTS_OF (CrewID, ShowID)

In these cases:

- No attribute determines another
- Every attribute is part of the key

BCNF holds trivially, therefore, **these relations are in BCNF.**

Conclusion

All relations satisfy the BCNF condition because:

- Either the determinant is a superkey, or
- There are no non-trivial functional dependencies

Therefore:

The database is fully normalized (BCNF, 3NF, 2NF, 1NF) and free from update, insertion, and deletion anomalies.

Top-3 Queries - SQL Text & Output

Query 1 - Upcoming Shows with Venue and Organizer

Question: Which upcoming fashion shows are scheduled, and who is responsible for each? Lists each show's theme alongside the hosting venue and the assigned organizer useful for logistics planning and stakeholder communication.

```
SELECT f.ShowID, f.Theme, v.Name AS venue, o.FullName AS organizer
FROM FASHION_SHOW f
JOIN VENUE v ON f.VenueID = v.VenueID
JOIN ORGANIZER o ON f.OrganizerID = o.OrganizerID
WHERE f.Date >= CURRENT_DATE;
```

Complexity Highlights:

Uses a **three-table JOIN**: FASHION_SHOW is the base, joined to VENUE on VenueID and to ORGANIZER on OrganizerID. The **WHERE f.Date >= CURRENT_DATE** filter restricts results to shows that have not yet passed, making the query dynamic and always current. Column aliases (**venue, organizer**) improve readability of the result set for non-technical stakeholders.

Output:

	Theme	Venue	Organizer
101	Summer Vibes	The Grand Ballroom	Amit Desai
102	Winter Luxe	Expo Convention Hall	Riya Mehta
103	Urban Street	Skyline Arena	Kunal Verma
104	Royal Heritage	Heritage Palace	Sneha Chauhan
105	Beach Style	Coastal View Hall	Kunal Verma
106	Modern Fusion	Metro Convention	Riya Mehta
107	Dubai Glamour	The Ritz Hall	Amit Desai

Interpretation: All 7 upcoming shows are returned with their respective venue and organizer. Kunal Verma and Riya Mehta each manage two shows, while Amit Desai manages two as well. Sneha Chauhan handles Royal Heritage exclusively. The query confirms no upcoming show is missing venue or organizer assignment.

Query 2 — Sponsors Whose Contribution Exceeds the Average

Business Question: Which sponsors have contributed more than the average total sponsorship amount across all sponsors? Identifies premium partners for preferential treatment and benefit negotiation.

```
SELECT s.SponsorID, s.CompanyName,
       SUM(sp.ContributionAmount) AS total
FROM SPONSOR s
JOIN SPONSORSHIP sp
     ON s.SponsorID = sp.SponsorID
GROUP BY s.SponsorID, s.CompanyName
HAVING SUM(sp.ContributionAmount) > (
    SELECT AVG(total_amount)
    FROM (
        SELECT SUM(ContributionAmount) AS total_amount
        FROM SPONSORSHIP
        GROUP BY SponsorID
    ) AS temp
);
```

Complexity Highlights:

The subquery in HAVING requires a **derived table** (inline view) to compute per-sponsor totals first, then averages over those totals - a two-level aggregation that cannot be expressed with a single GROUP BY.

Output:

SponsorID	CompanyName	Total Contribution
1	Luxe Brands Co.	162,000
2	GlowUp Cosmetics	104,000

Calculation: Average per sponsor = $(162,000 + 104,000 + 41,000 + 58,000) \div 4 = 91,250$.

Luxe Brands Co. and GlowUp Cosmetics both exceed this threshold, confirming their status as top-tier sponsors.

Query 3 – Shows Where Sponsorship Exceeded Total Garment Production Costs [Advanced]

Business Question: Which fashion shows were financially "in the green" i.e., total sponsorship received exceeded the combined production cost of all garments displayed in the show? Key profitability indicator for event planning.

```
SELECT f.ShowID, f.Theme
FROM FASHION_SHOW f
WHERE (
    SELECT SUM(sp.ContributionAmount)
    FROM SPONSORSHIP sp
    WHERE sp.ShowID = f.ShowID
) > (
    SELECT SUM(g.ProductionCost)
    FROM WALKS_IN w
    JOIN GARMENT g
      ON w.GarmentID = g.GarmentID
    JOIN SEGMENT s
      ON w.SegmentName = s.SegmentName AND w.ShowID = f.ShowID
    WHERE s.ShowID = f.ShowID
);
```

Complexity Highlights:

Uses **two independent subqueries** in the same WHERE clause - one aggregating across the SPONSORSHIP table, the other chaining three tables (WALKS_IN → GARMENT → SEGMENT) with a join. Both are re-evaluated per outer row, making this a classic correlated comparison pattern.

Output (final result - qualifying rows only):

ShowID	Theme	Sponsorship	Garment Costs	Status
101	Summer Vibes	120,000	63,000	Qualifies
102	Winter Luxe	63,000	75,300	Does not qualify
103	Urban Street	41,000	58,500	Does not qualify

Final Result: Only Show 101 "Summer Vibes" qualifies — its Rs.1,20,000 sponsorship comfortably covered the Rs.63,000 garment production costs, leaving a surplus of Rs.57,000.